

Redemption Architecture

Executive Summary

AWS · EKS · Terraform · Kubernetes · GitOps

CONFIDENTIAL · Internal Architecture Document

1 Executive Summary

This document outlines the architectural decisions, operational strategies, and team delegation plan for **The Redemption**, a business-critical point-deduction microservice on AWS. The design prioritises **zero-downtime**, rapid elasticity during **10× traffic spikes** (Flash Sales), and strict security boundaries using Infrastructure-as-Code (Terraform) and Kubernetes.

2 Architectural Decisions & Scalability Strategy

Compute & Architecture

- Microservice is containerised and hosted on **Amazon EKS**, spanning three Availability Zones for resilience against localised outages.
- An **AWS Application Load Balancer (ALB)** serves as the ingress point, routing traffic intelligently to healthy pods.

Handling 10× Flash Sales

- **Application Tier:** Horizontal Pod Autoscaler (HPA) monitors CPU and Memory metrics, automatically scaling the *redemption-api* deployment when utilisation crosses 70%.
- **Infrastructure Tier:** Karpenter / AWS Cluster Autoscaler detects pending pods and rapidly provisions right-sized EC2 instances, scaling back down to optimise costs.

State Buffering

- Incoming deduction requests are buffered through an **Amazon SQS FIFO queue** before being asynchronously committed to the multi-AZ Aurora PostgreSQL database, preventing connection exhaustion during flash sales.

3 Security & Networking

Network Isolation

Worker nodes and databases reside exclusively in private subnets with no direct internet ingress. EKS API endpoint access is restricted.

Least Privilege (IRSA)

IAM Roles for Service Accounts (IRSA) grants redemption-api pods granular, exact permissions — e.g. specific SQS queue access — avoiding broad EC2 instance profiles.

Encryption & Traffic Flow

Data is encrypted at rest via AWS KMS. Kubernetes Network Policies ensure the database can only be queried by authorised application pods.

4 Reliability & Observability

Self-Healing

- Kubernetes **liveness and readiness probes** automatically restart unhealthy containers.
- A **PodDisruptionBudget (PDB)** guarantees a minimum number of replicas remain available during voluntary disruptions or cluster upgrades.

Metrics & Tracing

- The *monitoring* namespace houses a fully automated **Prometheus & Grafana** stack with real-time dashboards for queue depth, pod resource usage, and API latency.
- **OpenTelemetry** is recommended for distributed tracing to identify bottlenecks during high load.

5 Architectural Trade-offs

EKS vs. Serverless (Fargate / Lambda)

While serverless removes node management, **EKS on EC2** was chosen for lower latency at high scale and absolute control over network routing and ingress — vital during intense flash sales.

Asynchronous Processing

SQS introduces **eventual consistency**: a user's point balance may take a few seconds to update during extreme spikes. This is traded off against the guarantee that the database won't crash, causing total service failure.

6 Day 2 Operations & Team Delegation

All deployments are managed via **GitOps (ArgoCD)** and Infrastructure-as-Code to minimise operational toil. The implementation is split across three engineers:

Role	Focus Area	Key Tasks
Senior Engineer (Lead)	Architecture, Security & IaC Foundation	• Core Terraform modules (VPC, EKS, IAM) • OIDC provider for IRSA • Network security boundaries • PR reviews for junior engineers

Role	Focus Area	Key Tasks
Junior Engineer 1 (App & Deploy)	Kubernetes Manifests & CI/CD	• Containerise redemption-api (Dockerfile) • K8s Deployments, Services, Ingress • GitHub Actions pipeline • ArgoCD synchronisation
Junior Engineer 2 (Observability)	Reliability & Monitoring	• HPA and PodDisruptionBudget (PDB) • Prometheus/Grafana via Helm • Dashboards & alerts (CPU, queue depth) • OpenTelemetry tracing setup